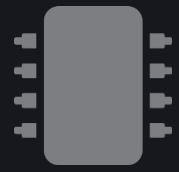
CSI101

Assembly Programming







Lectures 3 and 4 — x86 (IA-32) Inline Assembly with Visual C++

Dr. Kok Hoe WONG

All code targets Win32 Console Application · Visual Studio 2022 · x86 platform

What we will cover

Lecture 3 — Foundations

01 Introduction and motivation

02 Overview of x86 fundamentals

03 Why inline assembly?

04 MSVC _asm syntax basics

05 Registers and addressing

06 Data movement and arithmetic

07 Passing data between C and assembly

Lecture 4 — Applying it

08 Control flow

09 Functions and calling conventions

10 Limitations and compiler interaction

11 Practical applications

12 Debugging in Visual Studio 2022

13 Best practices

14 Summary and further reading

1

Introduction and motivation

Why a Year 1 computer scientist learns assembly at all.



You already know what happens. Now you write it.

WHAT YOU HAVE WRITTEN SO FAR

```
int total = 0;
for (int i = 1; i <= 10; i++)
    total += i;
```

WHAT THE PROCESSOR RECEIVES

```
    mov  eax, 0
    mov  ecx, 10
L1: add  eax, ecx
    loop L1
```



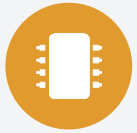
See what your code costs

A loop, a function call, an array index — each has a visible price in instructions.



Debug what you cannot otherwise see

When a crash lands in a library with no source, the disassembly is all you get.



Work where nothing else fits

Device drivers, boot code, embedded systems and interrupt handlers still need it.

Why learn assembly in the AI age?

You can only supervise a tool you understand better than it understands itself.

Catch AI's mistakes

Someone must spot a clobbered register or misaligned stack.

The foundation layer

Operating systems, compilers, and performance work all assume you understand this.

Hardest roles to automate

Embedded systems, security, and performance engineering reward this fluency.

Verifiable understanding

Small and mechanical enough to trace by hand — tool or no tool.

Where assembly sits

C++ source

What you write

```
total += i;
```

Assembly language

Readable names for machine instructions

```
add eax, ecx
```

Machine code

Binary the CPU decodes

```
01 C8
```

Hardware

Gates and transistors

```
the ALU adds two registers
```

Assembly is the last level a human can read, and the first the machine can almost run.

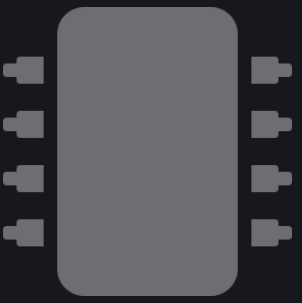
What you will be able to do

1. Write inline assembly inside a working C++ program, and build and run it in Visual Studio 2022.
2. Use the general-purpose registers and the common addressing modes correctly.
3. Move data between C++ variables and registers, and perform integer arithmetic.
4. Implement selection and iteration using compare instructions and conditional jumps.
5. Use the stack to pass arguments and return values across a function boundary.
6. Step through your own assembly in the debugger and read the register and flag state.

2

Overview of x86 fundamentals

The register set, the flags, memory and the stack.



IA-32: the model you are programming

WHAT IA-32 MEANS

- Intel Architecture, 32-bit — also written x86-32
- Registers are 32 bits wide; addresses are 32 bits
- So, one program can address 2^{32} bytes = 4 GB

WHY 32-BIT AND NOT 64-BIT?

- Microsoft's compiler supports inline assembly on x86 only
- On x64 the _asm keyword is rejected outright
- So, every project in this module builds for the x86 platform

LITTLE-ENDIAN — WORTH KNOWING NOW

x86 stores the least significant byte at the lowest address. The 32-bit value 0x12345678 held at address 0x1000 appears in memory as:



The general-purpose registers

EAX

Accumulator

Arithmetic, and the conventional place for a return value

EBX

Base

General use — but you must preserve it

ECX

Counter

Loop counts and shift counts; used automatically by LOOP

EDX

Data

Extends EAX for multiply and divide; general use otherwise

ESI

Source index

Source pointer for string and array operations

EDI

Destination index

Destination pointer for string and array operations

EBP

Base pointer

Points at the current stack frame — do not clobber

ESP

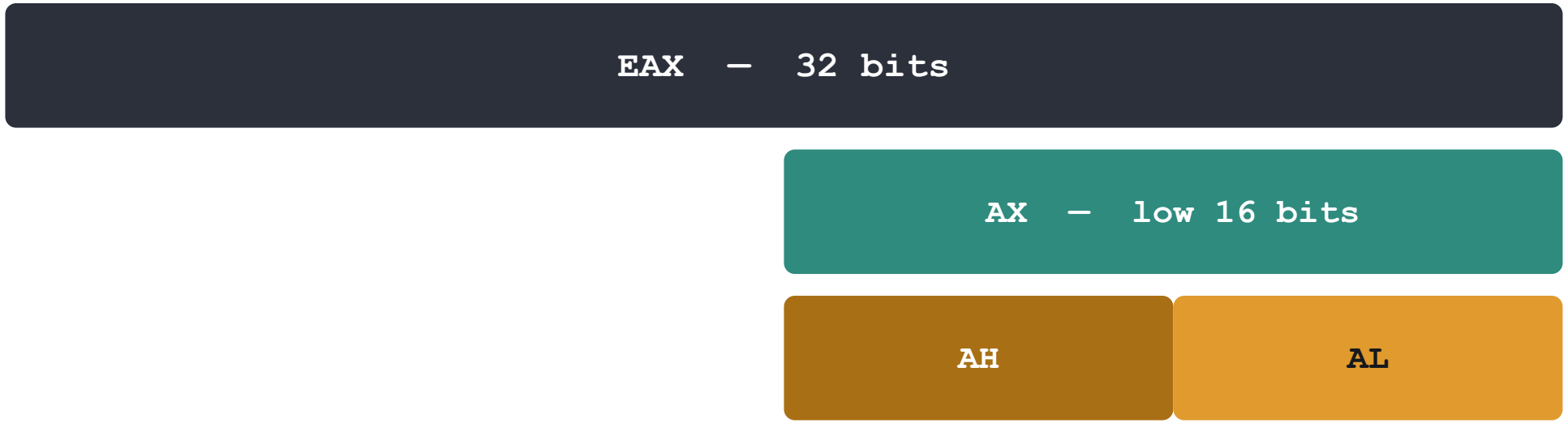
Stack pointer

Points at the top of the stack — do not clobber

“General-purpose” is only mostly true — the names describe what each was designed for, and some instructions still assume it.

Register sizes: EAX, AX, AH, AL

The first four registers can be accessed as 32, 16 or 8 bits. The smaller names are windows onto the same storage.



- Same pattern for EBX/BX/BH/BL, ECX and EDX
- ESI, EDI, EBP, ESP have 16-bit forms but no 8-bit forms

Writing to AL changes the bottom byte of EAX and leaves the top three bytes untouched. That is a common source of bugs.

The EFLAGS register

Arithmetic and comparison instructions set single-bit flags. Conditional jumps then read them. This is how decisions happen.

ZF

Zero

Set when the result was exactly zero

SF

Sign

Set when the result's top bit is 1 — negative in two's complement

CF

Carry

Set on unsigned overflow — a carry out of the top bit

OF

Overflow

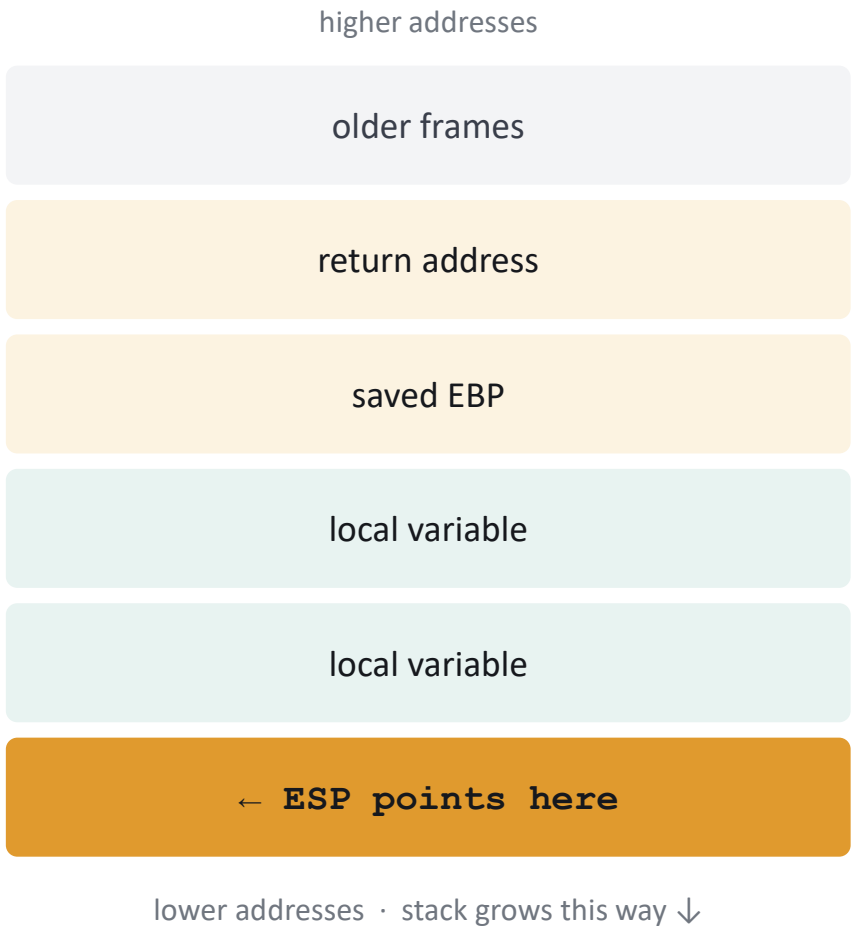
Set on signed overflow — the result will not fit

THE KEY IDEA

```
cmp   eax, ebx      ; subtracts ebx from eax, discards the result,
                    ; but keeps the flags it produced
je    equal_label   ; jumps only if ZF = 1
```

Memory and the stack

- The stack is a region of memory used for local variables, parameters and return addresses
- It grows downward — toward lower addresses
- ESP always points at the item currently on top
- PUSH subtracts 4 from ESP, then writes the value there
- POP reads the value at ESP, then adds 4 to ESP
- CALL pushes the return address; RET pops it back into the instruction pointer



Instruction anatomy

```
mov     eax, ebx        ; copy ebx into eax
```



Mnemonic

`mov`

The operation. Names the instruction: mov, add, cmp, jmp.



Destination

`eax`

Where the result goes. Written first in Intel syntax.



Source

`ebx`

Where the data comes from. Left unchanged.



Comment

`; ...`

Everything after a semicolon is ignored by the assembler.

Intel syntax reads right-to-left: destination ← source. Remember it as an assignment statement.

3

Why inline assembly?

Rather than a standalone .asm file assembled with MASM.



Two ways to write assembly on Windows

Inline assembly

`_asm blocks inside a .cpp file`

- No extra build configuration — it just compiles
- C++ handles input, output and data declaration
- Refer to C++ variables by their own names
- One file, one debugger session, one mental model
- Ideal for learning and for short critical sections

Standalone assembly

`a separate .asm file built with MASM`

- Must enable MASM in Build Customizations
- You write the I/O yourself, or import C library calls
- Full MASM directives, macros and data definitions
- Required for x64, where inline assembly does not exist
- Better for whole routines written entirely in assembly

This module uses inline assembly — you learn the instructions without also learning a build system.

The division of labour

Declare and print in C++. Compute in assembly. This is the pattern for every example in this module.

C++ DOES

- Variable and array declarations
- String literals for prompts and messages
- Anything involving the standard library
- The overall program structure

ASSEMBLY DOES

- Loading values into registers
- The arithmetic and the logic
- Comparisons and jumps
- Writing results back into C++ variables

4

MSVC _asm syntax basics

Writing your first block, and getting it to build.



Your first _asm block

```
#include <cstdio>

int main()
{
    int result;              // declared in C++

    _asm {
        mov     eax, 20
        add     eax, 22
        mov     result, eax  // write back to the C++ variable
    }

    printf("result = %d\n", result);
    return 0;
}
```

TWO SPELLINGS

_asm and __asm are the same keyword. __asm is the ISO-conformant spelling; _asm is accepted for compatibility and is used in your handout.

WHAT TO NOTICE

- result is declared by C++, used by name in asm
- No semicolons needed between instructions
- Both ; and // start a comment

Output: result = 42

Creating the project in Visual Studio 2022

1

Create the project

File > New > Project > Console App (C++). Give it a name and create it.

2

Switch the platform to x86

Use the Solution Platforms dropdown on the toolbar. It defaults to x64, where _asm will not compile.

3

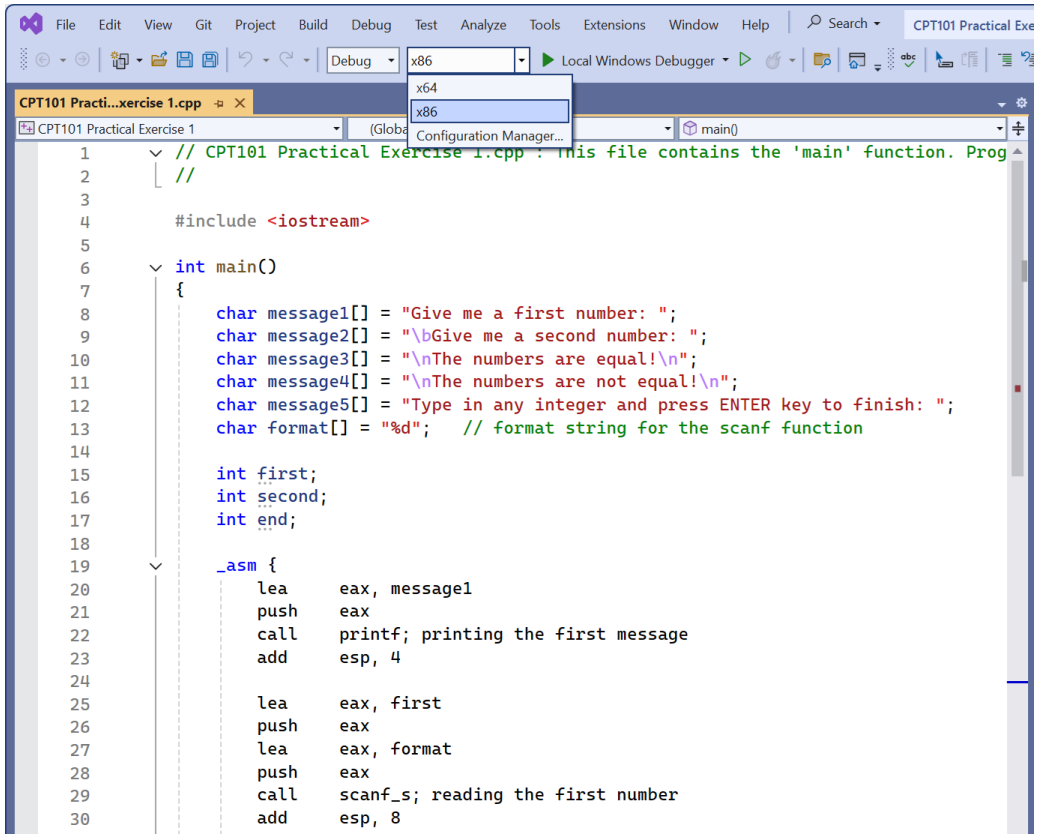
Write your code

Replace the generated main with your own. Include <cstdio> if you call printf or scanf_s.

4

Build and run

Ctrl + F5 runs without the debugger and keeps the console window open.



If you forget step 2, the compiler reports that the _asm keyword is not supported on this architecture (C4235).

Block forms and comments

BRACED BLOCK — USE THIS ONE

```
_asm {
    mov   eax, 1
    add   eax, 2
}
```

SINGLE INSTRUCTION

```
_asm mov eax, 1
_asm add eax, 2

// one keyword per line
```

COMMENTS — BOTH STYLES WORK

```
mov   eax, 1     ; assembler style
mov   ebx, 2     // C++ style
```

Prefer // in inline assembly. A stray ; inside a macro or a line continuation can confuse the C++ preprocessor.

Assembly is not case sensitive: MOV, mov and Mov are the same instruction. Pick lower case and stay consistent.

What you can and cannot write in a block

SUPPORTED

- All IA-32 instructions and Intel-syntax operands
- C and C++ variable, function and label names
- The PTR, OFFSET, TYPE, LENGTH and SIZE operators
- Preprocessor macros defined with #define
- Labels local to the enclosing function
- Structure and union member access with a dot

NOT SUPPORTED

- Data definition directives: DB, DW, DD — declare data in C++ instead
- MASM macro directives: MACRO, REPT, IRP, ENDM
- Segment and model directives: .MODEL, .DATA, .CODE
- The ALIGN and EVEN directives
- Any 64-bit instruction — this is a 32-bit target

The first item on the right is why every example declares its data in C++ and computes in assembly.

Reading the handout example

```
char message1[] = "Give me a first number: ";
char format[]   = "%d";
int  first;

_asm {
    lea     eax, message1   // eax = address of the string
    push    eax             // push the argument
    call    printf          // call the C library function
    add     esp, 4          // caller removes the argument

    lea     eax, first      // eax = address of the variable
    push    eax             // second argument to scanf_s
    lea     eax, format
    push    eax             // first argument to scanf_s
    call    scanf_s
    add     esp, 8          // two arguments, so 8 bytes
}
```

WHY LEA, NOT MOV?

LEA loads the address of something. MOV would load its contents. printf needs the address of the first character, not the character itself.

WHY PUSH IN REVERSE?

printf and scanf_s use the __cdecl convention: arguments are pushed right to left, and the caller cleans the stack afterwards with ADD ESP.

5

Registers and addressing inside _asm

Naming registers, and the ways to reach memory.



Operand types

Register

```
mov eax, ebx
```

Fastest. No memory access at all.

Immediate

```
mov eax, 42
```

A constant built into the instruction.

Memory

```
mov eax, total
```

A named C++ variable, or an address in brackets.

THE ONE RULE THAT CATCHES EVERYONE

```
mov  eax, total      // fine   — memory to register
mov  total, eax      // fine   — register to memory
mov  total, count    // ERROR  — both operands are memory
```

Addressing modes

Anything in square brackets means: go to that address and use what is there.

Direct	`mov eax, total`	The named variable’s own address
Register indirect	`mov eax, [esi]`	The address held in esi is used to access memory
Base + displacement	`mov eax, [esi + 8]`	Eight bytes past that address
Indexed with scale	`mov eax, [esi + ecx*4]`	Element ecx of an array of 4-byte items
Full form	`mov eax, [esi + ecx*4 + 8]`	Base, index, scale and displacement together

The scale factor may only be 1, 2, 4 or 8 — matching the size of a byte, short, int or double.

Working with a C++ array

```
int  numbers[5] = { 10, 20, 30, 40, 50 };
int  third;

_asm {
    lea     esi, numbers       // esi = address of numbers[0]
    mov     ecx, 2             // we want index 2
    mov     eax, [esi + ecx*4] // eax = numbers[2]
    mov     third, eax
}

// third == 30
```

LOAD THE BASE FIRST

LEA ESI, numbers puts the array's address in a register. Everything after that is arithmetic on that register — clear, and it always works.

WHY *4?

An int is four bytes. Index 2 sits eight bytes into the array. The scale factor does that multiplication as part of the address calculation.

Assembly has no bounds checking. Index 99 of a five-element array will read whatever happens to be there.

Sizes: BYTE, WORD and DWORD PTR

BYTE	8 bits	char
WORD	16 bits	short
DWORD	32 bits	int, long, pointer

You only need PTR when the size is genuinely ambiguous — usually when the destination is a bracketed address and the source is a constant.

```
int   value = 5;
char  letter = 'A';
int   arr[4];

_asm {
    mov  eax, value          // size known: DWORD
    mov  al,  letter         // size known: BYTE

    lea  esi, arr
    mov  DWORD PTR [esi], 7  // needed: [esi] alone
                             // does not say how big
}
```

Which registers must you protect?

FREE TO USE — NO NEED TO RESTORE

EAX **ECX** **EDX**

The compiler already assumes these may change across a call.

MUST BE SAVED AND RESTORED

EBX **ESI** **EDI**

Plus EBP and ESP, which you must never disturb at all.

```
_asm {
    push    ebx             // save before use
    push    esi
    lea     esi, numbers    // now safe to use them
    mov     ebx, [esi]
    pop     esi             // restore in reverse order
    pop     ebx
}
```

6

Data movement and arithmetic

The instructions you will use in almost every program.



Moving data

MOV	`mov eax, ebx`	Copy source to destination. Source unchanged.
LEA	`lea esi, arr`	Load the address of something, not its contents.
XCHG	`xchg eax, ebx`	Swap two operands in one instruction.
PUSH	`push eax`	Decrement ESP by 4, then store the value on the stack.
POP	`pop eax`	Read the top of the stack, then increment ESP by 4.
MOVZX	`movzx eax, bl`	Copy a smaller value, filling the top with zeros.
MOVSX	`movsx eax, bl`	Copy a smaller value, preserving the sign bit.

MOVZX and MOVSX exist because a plain MOV cannot change size. Use MOVSX for signed values, MOVZX for unsigned.

Integer arithmetic

ADD

```
add eax, ebx
```

eax = eax + ebx

SUB

```
sub eax, ebx
```

eax = eax − ebx

INC

```
inc eax
```

eax = eax + 1

DEC

```
dec eax
```

eax = eax − 1

NEG

```
neg eax
```

eax = −eax (two's complement)

IMUL

```
imul eax, ebx
```

Signed multiply, result in eax

DIVISION IS DIFFERENT — READ CAREFULLY

```
mov  eax, 17      // dividend goes in eax
cdq               // sign-extend eax into edx  (use xor edx,edx if unsigned)
idiv ebx          // eax = quotient, edx = remainder
```

Logic and shifts

BITWISE LOGIC

AND	`and eax, 0Fh`	keep only the bits set in both
OR	`or eax, 80h`	set the bits present in either
XOR	`xor eax, eax`	a fast way to zero a register
NOT	`not eax`	invert every bit

SHIFTS AND ROTATES

SHL	`shl eax, 3`	left shift — multiply by 8
SHR	`shr eax, 1`	logical right — zeros come in
SAR	`sar eax, 1`	arithmetic right — sign preserved
ROL / ROR	`rol eax, 4`	rotate: bits wrap round

SHR AND SAR ARE NOT INTERCHANGEABLE

```
// starting from  -32  (0xFFFFFFE0)
shr  eax, 2       // 0x3FFFFFF8 = 1073741816   — sign bit lost
sar  eax, 2       // 0xFFFFFFF8 = -8           — correct division by 4
```

Worked example: arithmetic on C++ variables

```
int a = 17, b = 5;
int sum, diff, prod, quot, rem;

_asm {
    mov     eax, a
    add     eax, b
    mov     sum, eax        // 22
    mov     eax, a
    sub     eax, b
    mov     diff, eax       // 12
    mov     eax, a
    imul    eax, b
    mov     prod, eax       // 85
    mov     eax, a
    cdq                     // sign-extend into edx
    idiv    b
    mov     quot, eax       // 3
    mov     rem,  edx       // 2
}
```

THE PATTERN

Load into a register, operate, store back. Every one of these five calculations follows exactly that shape.

OUTPUT

```
22 12 85 3 2
```

Note that IDIV takes the variable b directly as its operand — a memory operand is allowed there.

7

Passing data between C and assembly

Variables, arrays, structures and the library calls.



Using C++ names inside a block

```
struct Point { int x; int y; };

int    count = 3;
int    values[3] = { 5, 10, 15 };
Point  p = { 7, 9 };
int    result;

_asm {
    mov     eax, count          // simple variable
    lea     esi, values         // address of an array
    mov     ebx, [esi + 4]      // values[1]
    mov     ecx, p.x            // structure member by name
    add     eax, ebx
    add     eax, ecx
    mov     result, eax         // 3 + 10 + 7 = 20
}
```

THE COMPILER RESOLVES THE NAME

You write the variable's name; the compiler substitutes its actual address or stack offset. You never need to know where it lives.

SCOPE RULES STILL APPLY

Only names visible at that point in the C++ code can be used. Names are case sensitive here, even though instruction mnemonics are not.

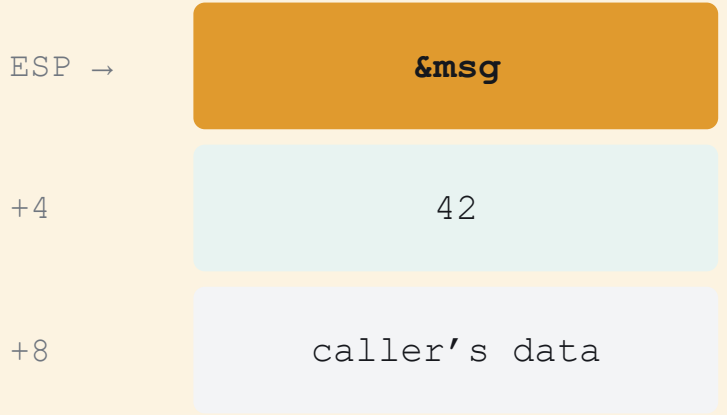
Calling C library functions

```
char msg[] = "Value is %d\n";
int  value = 42;

_asm {
    mov     eax, value
    push    eax             // last argument pushed first
    lea     eax, msg
    push    eax             // format string pushed last
    call    printf
    add     esp, 8          // two arguments = 8 bytes
}
```

Push arguments right to left. Clean up 4 bytes per argument afterwards. Forgetting the cleanup corrupts the stack.

STACK DURING THE CALL



printf reads the format string from the top of the stack, sees %d, and takes the next four bytes as the integer.

ADD ESP, 8 then throws both away.

Reading input with scanf_s

```
char format[] = "%d";
int  number;

_asm {
    lea     eax, number    // address of the destination
    push    eax
    lea     eax, format    // address of the format string
    push    eax
    call    scanf_s
    add     esp, 8
}
```

For a string, scanf_s does need a buffer size — push it as an extra argument and clean up 12 bytes instead of 8.

ADDRESS, NOT VALUE

scanf_s must write into your variable, so it needs the variable's address. That is why LEA appears rather than MOV.

WHY SCANF_S, NOT SCANF?

Visual Studio rejects plain scanf as unsafe (warning C4996). For %d no extra size argument is required, so the call takes exactly two pushes.

End of Lecture 3

You can now write, build and run a program whose calculations happen in assembly.

SO FAR

- Registers, flags, memory and the stack
- The _asm block and what it may contain
- Addressing modes and operand sizes
- Data movement and integer arithmetic
- Talking to C++ variables and library calls

NEXT TIME

- Control flow: comparisons, jumps and loops
- Functions, the stack frame and calling conventions
- What the compiler will and will not let you do
- Five worked applications — your lab exercises
- Debugging assembly in Visual Studio

8

Control flow in inline assembly

Comparisons, conditional jumps, and building loops by hand.



There is no if. There is no while.

WHAT YOU WRITE IN C++

```
if (a > b)
    max = a;
else
    max = b;
```

WHAT IT BECOMES

```
        mov  eax, a
        cmp  eax, b
        jle  use_b
        jmp  done
use_b:  mov  eax, b
done:   mov  max, eax
```

Every selection and every loop you have ever written is a comparison followed by a conditional jump. That is all the hardware offers.

1 — Compare
CMP sets the flags

2 — Jump
A conditional jump reads them

3 — Label
Execution continues from there

Conditional jumps: signed and unsigned

The same bit pattern means different things depending on whether you treat it as signed. So, there are two families of jumps.

SIGNED — USE WITH INT

`JG / JNLE`	greater than
`JGE / JNL`	greater or equal
`JL / JNGE`	less than
`JLE / JNG`	less or equal

UNSIGNED — USE WITH ADDRESSES AND UNSIGNED

`JA / JNBE`	above
`JAE / JNB`	above or equal
`JB / JNAE`	below
`JBE / JNA`	below or equal

Equality is shared: JE / JZ jumps if equal, JNE / JNZ jumps if not equal. Also, JMP (always), JS / JNS (sign), JC / JNC (carry).

Building a loop by hand

COUNTED LOOP — TEST AT THE BOTTOM

```
        mov   ecx, 5          // counter
        mov   eax, 0          // running total
again:  add   eax, ecx
        dec   ecx
        jnz   again           // repeat while ecx != 0
        mov   total, eax      // 5+4+3+2+1 = 15
```

CONDITIONAL LOOP — TEST AT THE TOP

```
        mov   esi, 0
check:  cmp   esi, limit
        jge   finish          // leave if esi >= limit
        inc   esi
        jmp   check
finish:
```

THE PATTERN BEHIND BOTH

1. Initialise
set up the counter before the label

2. Label
mark where the loop body begins

3. Body
do the work

4. Update and test
change the counter, then jump back or fall through

The LOOP instruction

LOOP DOES TWO THINGS AT ONCE

- Decrements ECX
- Jumps to the label if ECX is not zero
- ECX is not optional — LOOP always uses it
- It does not change any flags

```
        mov   eax, 0
        mov   ecx, 10        // loop 10 times
again:  add   eax, ecx
        loop  again          // dec ecx, jump if != 0
        mov   total, eax     // 55
```

TWO TRAPS

ECX = 0 on entry

LOOP decrements first, so ECX wraps to 4,294,967,295 and the loop runs about four billion times. Test for zero before you start.

Short jump only

The target must be within 128 bytes. A long loop body produces a jump-out-of-range error at build time.

Nested loops: save the outer counter

```
_asm {
        mov     ecx, 3              // outer count
outer:  push    ecx                 // save it

        mov     ecx, 4              // inner count reuses ecx
inner:  // ... body of the inner loop ...
        loop    inner

        pop     ecx                 // restore the outer count
        loop    outer
}
```

THE PROBLEM

Both loops want ECX. Without saving it, the inner loop destroys the outer count, and the outer loop never terminates correctly.

THE FIX

PUSH before the inner loop, POP after it. Every push must have exactly one matching pop on every path through the code.

A jump that skips over a POP leaves the stack unbalanced. That is one of the hardest bugs in assembly to find.

TEST and the sign of a value

CMP VS TEST

CMP subtracts and keeps the flags — use it to order two values.

TEST performs a bitwise AND and keeps the flags — use it to ask whether particular bits are set.

```
test   eax, eax      // fastest zero test
jz     is_zero

test   eax, 1        // is the bottom bit set?
jnz    is_odd
```

WORKED EXAMPLE — ABSOLUTE VALUE WITHOUT A BRANCH ON THE VALUE ITSELF

```
        mov    eax, n
        test   eax, eax      // sets SF from the sign bit
        jns    positive      // jump if not signed
        neg    eax           // eax = -eax
positive:
        mov    result, eax
```

9

Functions and calling conventions

The stack frame, and the contract between caller and callee.



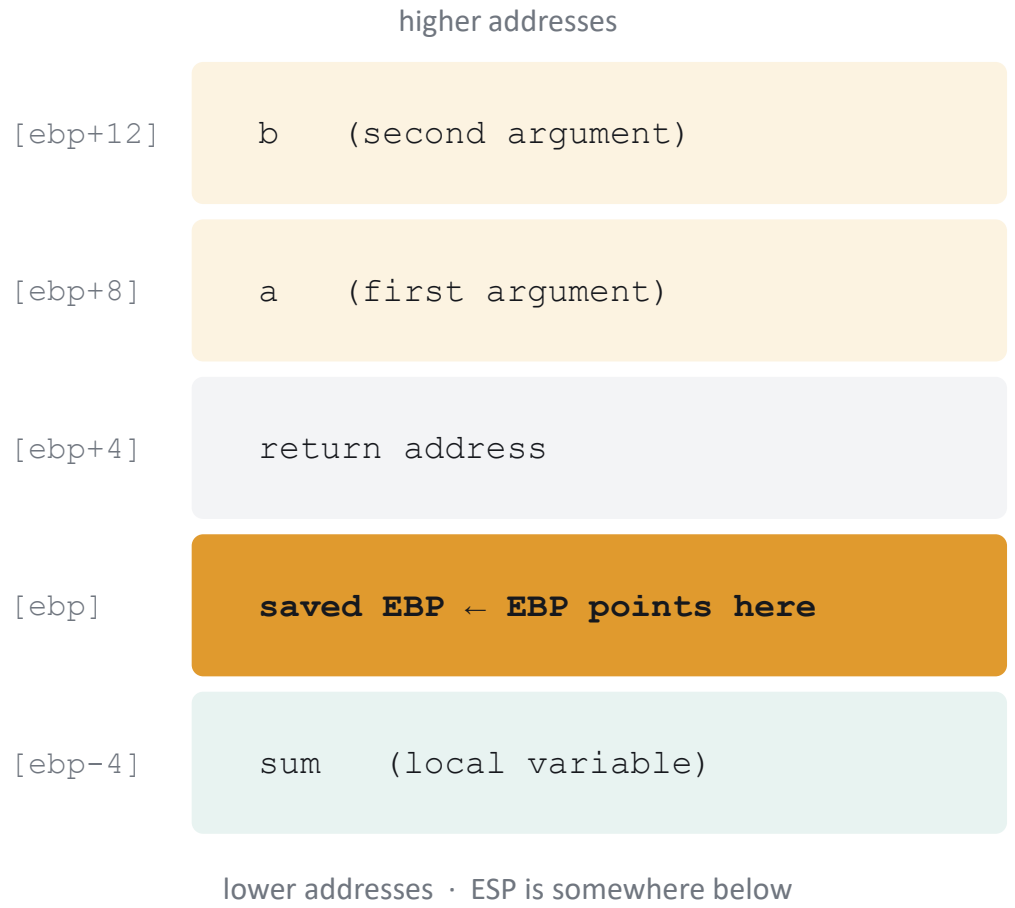
What a call actually does

1	**Caller pushes arguments**	In __cdecl, right to left, four bytes each.
2	**CALL pushes the return address**	The address of the instruction after the CALL, then jumps.
3	**Callee builds a stack frame**	PUSH EBP; MOV EBP, ESP — now EBP is a fixed reference point.
4	**Callee does its work**	Arguments at [EBP+8] and up; locals below EBP.
5	**Callee tears the frame down**	MOV ESP, EBP; POP EBP — restores the caller's frame.
6	**RET pops the return address**	Execution resumes in the caller. Result is left in EAX.
7	**Caller removes the arguments**	ADD ESP, n — in __cdecl this is the caller's job.

The stack frame

```
int add(int a, int b)
{
    int sum;
    _asm {
        mov  eax, a      // [ebp+8]
        add  eax, b      // [ebp+12]
        mov  sum, eax    // [ebp-4]
    }
    return sum;
}
```

Write a, b and sum by name. The compiler turns each into the right offset from EBP for you.



Three calling conventions

`__cdecl`

The C++ default

Arguments pushed right to left. The CALLER removes them. Required for variadic functions like printf.

`__stdcall`

The Windows API

Arguments pushed right to left. The CALLEE removes them with RET n. Slightly smaller code.

`__fastcall`

Speed-oriented

First two integer arguments go in ECX and EDX. Anything further is pushed. The callee cleans up.

All three return an integer result in EAX. A 64-bit result comes back in EDX:EAX. If you call a function with the wrong convention, the stack is left corrupted and the program usually crashes some time later.

Writing a function with an _asm body

```
int maxOf(int a, int b)
{
    int result;

    _asm {
        mov     eax, a
        cmp     eax, b
        jge     done        // a >= b, keep it
        mov     eax, b
    done:
        mov     result, eax
    }

    return result;          // clear and portable
}
```

RECOMMENDED STYLE

Store the answer in a local variable and return it with a normal C++ return statement. The intent is obvious to any reader.

THE ALTERNATIVE

MSVC also returns whatever is left in EAX if the function has no return statement. It works, but it is easy to misread — prefer the explicit form.

Note the label done: — labels live inside the function, so two functions may each have a label of the same name.

Calling your own function from assembly

```
int addTwo(int a, int b) { return a + b; }

int main()
{
    int answer;
    _asm {
        push    7               // second argument
        push    5               // first argument
        call    addTwo
        add     esp, 8          // __cdecl: caller cleans up
        mov     answer, eax     // result comes back in eax
    }
    // answer == 12
}
```

PUSH IN REVERSE

The last argument is pushed first, so the first argument ends up nearest the top of the stack — where the callee expects it at [ebp+8].

DO NOT FORGET THE CLEANUP

Two arguments means ADD ESP, 8. Omit it and the stack drifts a little further out of balance on every call.

The C++ function name is written directly. The compiler resolves it to the right address at link time.

10

Limitations and compiler interaction



What inline assembly costs you, and what it forbids.

The hard limitations



x86 only

MSVC supports _asm on 32-bit x86 and nowhere else. On x64 or ARM the keyword is rejected at compile time.



Optimisation is restricted

A function containing an _asm block will not be inlined, and the optimiser treats the block as opaque.



Not portable

The code is tied to one compiler, one architecture and one operating system.



No MASM directives

No data definitions, macros or segment directives. Declare all data in C++.

For production x64 code the modern answer is compiler intrinsics, or a separate .asm file assembled with ml64.

The rules you must obey

Preserve EBX, ESI and EDI	If your block changes them, push on entry and pop on exit.
Never disturb EBP or ESP	The compiler uses them to find your variables. Corrupt them and everything breaks.
Leave the stack balanced	Every PUSH needs exactly one POP, on every path through the block.
Clear the direction flag	If you use STD, restore it with CLD before the block ends.
Leave the FPU stack clean	If you use floating-point instructions, pop everything you pushed.
Do not jump out of a block	Jumping into or out of an _asm block from C++ code is not supported.

Break any of these and the failure usually appears somewhere else entirely.

How the compiler sees your block

WHAT IT DOES UNDERSTAND

- Variable names, and where each one lives
- Function names and their addresses
- That the block might change EAX, ECX and EDX

WHAT IT DOES NOT

- Whether your logic is correct
- Whether you balanced the stack
- Whether you restored EBX, ESI and EDI

A CONSEQUENCE WORTH KNOWING

Because the optimiser cannot see through an _asm block, hand-written assembly is often SLOWER than the code the compiler would have produced. A modern optimiser schedules instructions, allocates registers and vectorises loops better than almost anyone can by hand.

11

Practical applications

Five complete programs — and your five lab exercises.



The five exercises

11.1	**Compare two numbers**	Read two integers and report whether they are equal, or which is larger.
11.2	**Logical and arithmetic shift**	Show why SHR and SAR give different answers for a negative value.
11.3	**ECX as an automatic loop counter**	Use the LOOP instruction to accumulate a total.
11.4	**A function using the stack**	Pass arguments, branch on a condition, return a result in EAX.
11.5	**Selection sort and bubble sort**	Sort an array in place with nested loops and indexed addressing.

Each one becomes a lab exercise. Today you see it work; in the lab you extend it.

11.1 Compare two numbers

THE TASK

- Prompt for two integers and read them in
- Compare them
- Report equal, first larger, or second larger

IDEAS IT EXERCISES

- Calling printf and scanf_s from assembly
- CMP followed by JE and JG
- Converging branches onto a single exit

EXPECTED OUTPUT

```
Enter the first number: 17
Enter the second number: 9

The first number is larger.
```

11.1 Declarations and input

```
#include <cstdio>

char msgFirst[]     = "Enter the first number: ";
char msgSecond[]    = "Enter the second number: ";
char msgEqual[]     = "\nThe numbers are equal.\n";
char msgFirstBig[]  = "\nThe first is larger.\n";
char msgSecondBig[] = "\nThe second is larger.\n";
char format[]       = "%d";
int  first, second;
_asm {
    lea     eax, msgFirst
    push    eax
    call    printf
    add     esp, 4
    lea     eax, first      // address to write into
    push    eax
    lea     eax, format
    push    eax
    call    scanf_s
    add     esp, 8
    // ... repeat for msgSecond and second ...
}
```

ALL DATA IN C++

Every string and variable is declared by C++. Inline assembly cannot define data, so this split is not a style choice — it is required.

COUNT THE CLEANUP

printf here takes one argument, so ADD ESP, 4.

scanf_s takes two, so ADD ESP, 8.

Four bytes per argument, every time.

11.1 Comparing and branching

```
    mov     eax, first
    cmp     eax, second
    je      equal           // ZF set  -> equal
    jg      firstBigger     // signed greater than

    lea     eax, msgSecondBig
    jmp     show

firstBigger:
    lea     eax, msgFirstBig
    jmp     show

equal:
    lea     eax, msgEqual

show:
    push    eax
    call    printf
    add     esp, 4
}
```

ONE EXIT, THREE PATHS

Each branch loads a different address into EAX and jumps to show. The printing happens once, in one place. Less code, fewer mistakes.

WHY JG AND NOT JA?

These are signed ints, so the signed jump is correct. With JA, any negative number would appear to be the larger one.

Lab extension: add a third number, or loop until the user enters two equal values.

11.2 Logical and arithmetic shift

THE TASK

- Take a negative integer
- Shift it right two places, twice — once with SHR, once with SAR
- Print both results and explain the difference

THE IDEA

Shifting right divides by a power of two — but only if the sign bit is handled. SHR always shifts in zeros; SAR copies the existing sign bit.

start	1111 1111 1111 1111 1111 1111 1110 0000	−32
shr 2	0011 1111 1111 1111 1111 1111 1111 1000	1 073 741 816
sar 2	1111 1111 1111 1111 1111 1111 1111 1000	−8

Only SAR gives the arithmetically correct answer: −32 ÷ 4 = −8.

11.2 The program

```
#include <cstdio>

int value = -32;
int logical, arithmetic, leftShift;

_asm {
    mov     eax, value
    shr     eax, 2        // logical - zeros shifted in
    mov     logical, eax

    mov     eax, value
    sar     eax, 2        // arithmetic - sign kept
    mov     arithmetic, eax

    mov     eax, value
    shl     eax, 1        // left shift = times 2
    mov     leftShift, eax
}

printf("shr : %d\nsar : %d\nshl : %d\n", logical, arithmetic, leftShift);
```

OUTPUT

```
shr : 1073741816
sar : -8
shl : -64
```

THE RULE

Use SAR for signed values.
Use SHR for unsigned values.

Left shift is the same instruction either way — SHL and SAL are two names for it.

Lab extension: shift by a count held in CL rather than a constant, and test with both positive and negative inputs.

11.3 ECX as an automatic loop counter

THE TASK

Add the integers from 1 to 10 using the LOOP instruction, with no explicit comparison and no explicit decrement.

```
mov  ecx, 10
sumLoop:
  add  eax, ecx
  loop sumLoop
```

TRACING IT

ECX 10	ECX 9	ECX 8	ECX …	ECX 2	ECX 1
EAX 10	EAX 19	EAX 27	EAX …	EAX 54	EAX 55

LOOP decrements ECX and jumps back while it is not zero. When ECX reaches 0 the loop falls through, and EAX holds 55.

11.3 Two versions

SUM 1 TO 10

```
int total;

_asm {
    xor     eax, eax        // total = 0
    mov     ecx, 10         // counter
sumLoop:
    add     eax, ecx
    loop    sumLoop
    mov     total, eax      // 55
}
```

SUM AN ARRAY

```
#define SIZE 5
int data[SIZE] = { 4, 8, 15, 16, 23 };
int total;

_asm {
    push    esi
    lea     esi, data
    xor     eax, eax
    mov     ecx, SIZE
nextItem:
    add     eax, [esi]
    add     esi, 4          // next int
    loop    nextItem
    mov     total, eax      // 66
    pop     esi
}
```

Note the right-hand version saves and restores ESI, and uses #define rather than const int — a preprocessor macro becomes a plain number the assembler can use.

11.4 A function that uses the stack

Write absDiff(a, b) returning the absolute difference between two integers — exercising stack parameters, a jump on the sign flag, saving EBX, and returning a result in EAX.

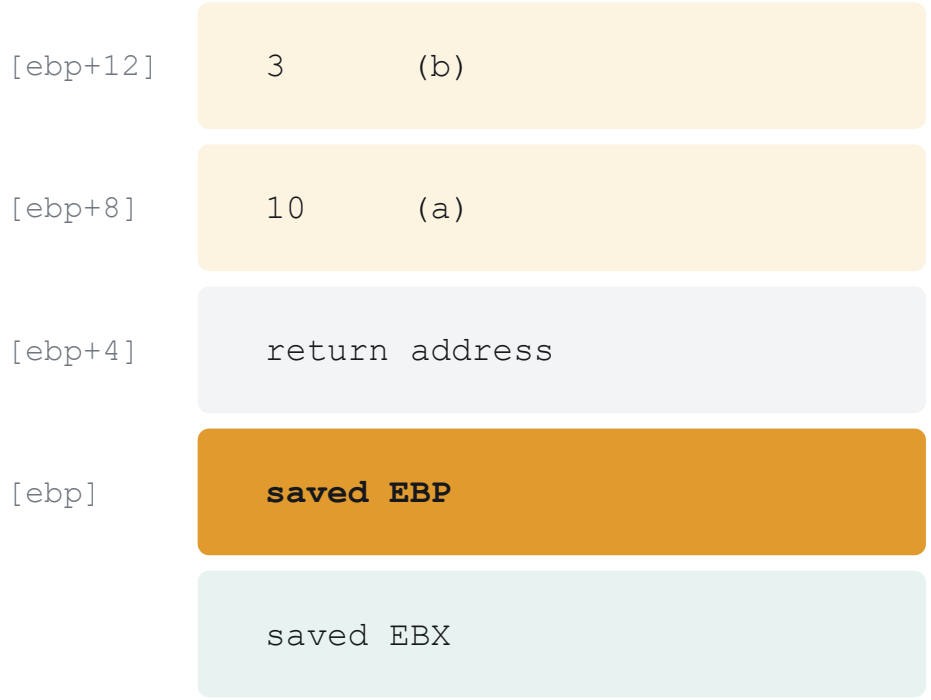
```
int absDiff(int a, int b)
{
    int result;
    _asm {
        push    ebx             // preserve ebx before using it
        mov     eax, a          // parameter at [ebp+8]
        mov     ebx, b          // parameter at [ebp+12]
        sub     eax, ebx
        jns     done            // SF clear -> already positive
        neg     eax             // otherwise flip the sign
    done:
        mov     result, eax
        pop     ebx             // restore on every path
    }
    return result;
}
```

11.4 Calling it, and what the stack looks like

```
int main()
{
    int answer;
    _asm {
        push    3               // second argument
        push    10              // first argument
        call    absDiff
        add     esp, 8          // caller cleans up
        mov     answer, eax     // result arrives in eax
    }
    printf("|10 - 3| = %d\n", answer);   // 7
}
```

Pushed 3 then 10, so 10 sits nearer the top — exactly where the callee looks for its first parameter.

Stack immediately after CALL



Lab extension: add a third parameter, or return the result via a pointer argument instead of EAX.

11.5 Two sorting algorithms

Bubble sort

- Compare each adjacent pair; swap them if they are out of order
- Repeat until a whole pass makes no swaps
- Each pass floats the largest remaining value to the end
- Many swaps, simple comparisons

Selection sort

- Scan the unsorted part for the smallest value
- Swap it into the first unsorted position
- Move the boundary along and repeat
- Few swaps — at most one per pass

Both are $O(n^2)$. Both need nested loops, indexed addressing and a swap — which is exactly why they are good assembly exercises.

11.5 Bubble sort in inline assembly

```
#define N 8
int arr[N] = { 42, 7, 19, 3, 88, 25, 1, 56 };

_asm {
    push    ebx
    push    esi
    push    edi

    lea     edi, arr      // edi = base address
    mov     ecx, N
    dec     ecx           // N-1 passes

outerLoop:
    push    ecx           // save pass counter
    xor     esi, esi      // i = 0
```

```
innerLoop:
    mov     eax, [edi + esi*4]      // arr[i]
    mov     ebx, [edi + esi*4 + 4]  // arr[i+1]
    cmp     eax, ebx
    jle     noSwap        // already ordered
    mov     [edi + esi*4], ebx
    mov     [edi + esi*4 + 4], eax
noSwap:
    inc     esi
    dec     ecx
    jnz     innerLoop

    pop     ecx           // restore pass counter
    loop    outerLoop

    pop     edi
    pop     esi
    pop     ebx
}
```

Both loops want ECX — PUSH it before the inner loop, POP it after. ECX shrinks each pass, so the sorted tail is skipped.

11.5 Watching the first pass

Starting array: 42 7 19 3 88 25 1 56

compare 42 and 7	**swap**	7 42 19 3 88 25 1 56
compare 42 and 19	**swap**	7 19 42 3 88 25 1 56
compare 42 and 3	**swap**	7 19 3 42 88 25 1 56
compare 42 and 88	**keep**	7 19 3 42 88 25 1 56
compare 88 and 25	**swap**	7 19 3 42 25 88 1 56
compare 88 and 1	**swap**	7 19 3 42 25 1 88 56
compare 88 and 56	**swap**	7 19 3 42 25 1 56 88

After one pass the largest value, 88, has reached the end. The next pass needs only seven of the eight positions.

11.5 Selection sort in inline assembly

```
#define N 8
int arr[N] = { 42, 7, 19, 3, 88, 25, 1, 56 };

_asm {
    push    ebx
    push    esi
    push    edi

    lea     edi, arr
    xor     ecx, ecx        // i = 0

outerLoop:
    mov     edx, ecx        // minIndex = i
    mov     esi, ecx
    inc     esi             // j = i + 1

innerLoop:
    cmp     esi, N
    jge     endInner
    mov     eax, [edi + esi*4]   // arr[j]
    cmp     eax, [edi + edx*4]   // vs arr[min]
    jge     notSmaller
    mov     edx, esi             // new minimum
notSmaller:
    inc     esi
    jmp     innerLoop
endInner:                        // swap i and min
    mov     eax, [edi + ecx*4]
    mov     ebx, [edi + edx*4]
    mov     [edi + ecx*4], ebx
    mov     [edi + edx*4], eax
    inc     ecx
    mov     eax, N
    dec     eax
    cmp     ecx, eax
    jl      outerLoop       // while i < N-1
    pop     edi
    pop     esi
    pop     ebx
}
```

Register roles: EDI base · ECX i · ESI j · EDX index of smallest · EAX / EBX values being compared

11.5 Comparing the two

BUBBLE SORT	
Comparisons	$\frac{n(n-1)}{2}$ in the worst case
Swaps	Up to $\frac{n(n-1)}{2}$ — many
Best case	O(n) if you add an early exit
In assembly	Simple body; needs the ECX push/pop

SELECTION SORT	
Comparisons	Always $\frac{n(n-1)}{2}$
Swaps	At most $(n-1)$ — few
Best case	Same as worst — no early exit
In assembly	More index juggling; no nested ECX clash

LAB EXTENSIONS

Sort in descending order · add an early-exit flag to bubble sort · count and print the number of swaps · sort an array of characters instead of integers.

12

Debugging inline assembly

Seeing the registers, the flags and the stack as your code runs.



Turn on address-level debugging first

1

Enable the setting

Tools > Options > Debugging > General. Tick “Enable address-level debugging”. Without it the assembly windows stay empty.

2

Set a breakpoint

Click in the left margin next to the _asm block, or press F9 on that line.

3

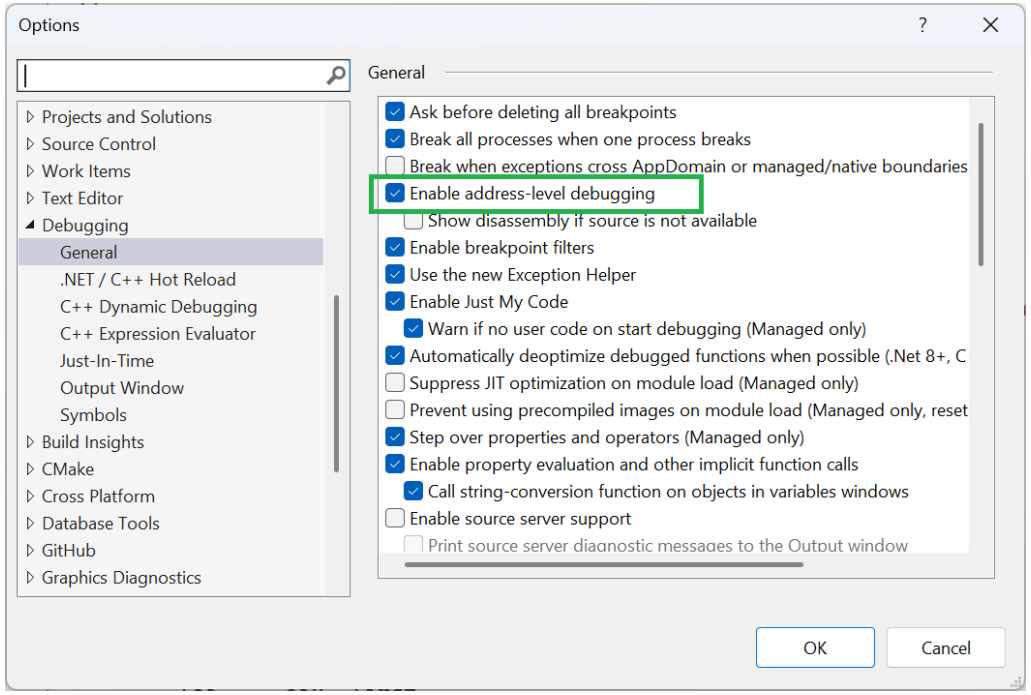
Start debugging

F5. Execution stops at the breakpoint and the debug windows become available.

4

Open the windows you need

Debug > Windows > Disassembly, Registers, Memory and Watch.

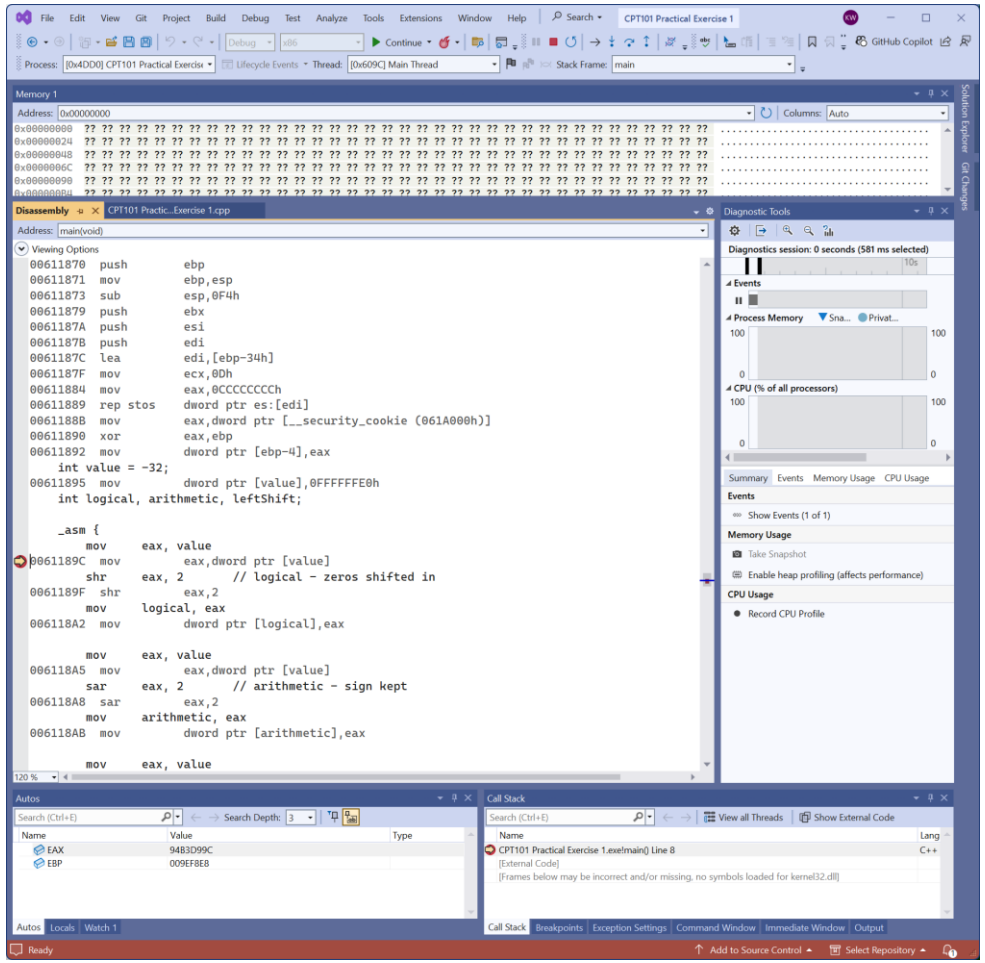


Enable address-level debugging in Visual Studio

The Disassembly window

- Debug > Windows > Disassembly, or Ctrl + Alt + D while stopped at a breakpoint
- Shows your instructions, their addresses and the machine code bytes they became
- A yellow arrow marks the instruction about to execute
- Right-click to toggle source code, address and byte columns
- F11 now steps one machine instruction at a time, not one C++ line

```
00E51A3C    8B 45 FC    mov  eax, dword ptr [ebp-4]
00E51A3F    C1 E8 02    shr  eax, 2
```



Disassembly window, stopped inside an _asm block, showing addresses, opcode bytes, instructions and the yellow current-instruction arrow.

Registers, flags and memory

REGISTERS WINDOW

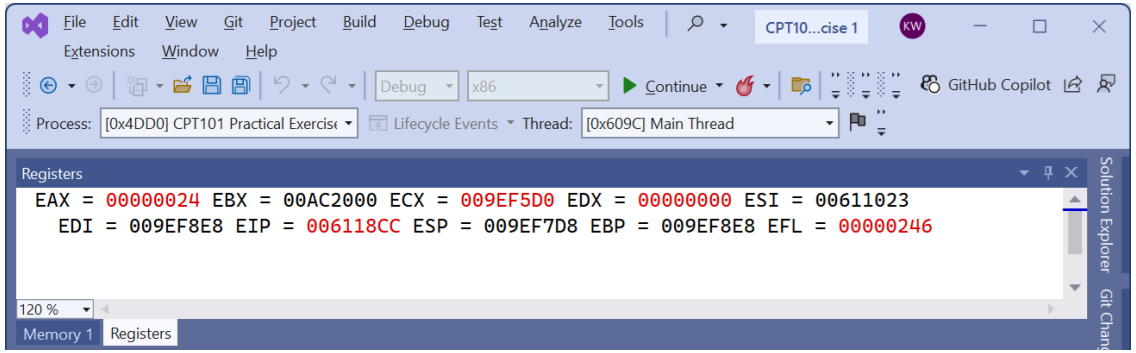
- Debug > Windows > Registers, or Ctrl + Alt + G
- Values that changed on the last step appear in red
- Right-click to add the Flags group

READING THE FLAGS

They appear as short names with a value: ZR for zero, CY for carry, OV for overflow, PL or NG for sign. Watch them change as you step over a CMP.

MEMORY WINDOW

- Debug > Windows > Memory > Memory 1
- Type a variable name or &arr in the Address box
- Remember little-endian: bytes appear reversed
- Set Columns to 4 to line an int array up neatly



Registers window showing EAX–EDI and the flag group, with recently changed values highlighted in red.

Stepping and watching

F9	Toggle a breakpoint
F5	Start or continue
F10	Step over
F11	Step into — one instruction in Disassembly
Ctrl + Alt + D	Disassembly window
Ctrl + Alt + G	Registers window

WATCH WINDOW TRICKS

Type a register name into the Watch window:

```
@eax        the register's value
arr,8       first 8 elements of an array
arr[2]      one element by index
```

A GOOD ROUTINE

- Watch the array in the Watch window
- Breakpoint on the swap inside the loop
- Step and confirm each swap does what you expect

Common errors and what they mean

`_asm not supported on this architecture`	The platform is x64. Switch the Solution Platforms dropdown to x86.
`Improper operand type / operand size conflict`	The operand size is ambiguous. Add DWORD PTR, or check you are not mixing eax with al.
`Undeclared identifier in an _asm block`	A misspelt variable name, or one not in scope. Names are case sensitive here.
`The program hangs and never finishes`	A loop whose exit condition is never met — often LOOP entered with ECX already zero.
`Crash after the _asm block, not inside it`	The stack is unbalanced, or EBX, ESI, EDI or EBP was not restored.
`Right answer, wrong sign`	SHR used where SAR was needed, or an unsigned jump used on signed data.

The last two are the dangerous ones: the program builds, runs, and quietly does the wrong thing.

13

Best practices

Habits that make assembly readable and keep it correct.



Writing assembly people can read

Comment the intent, not the instruction

“; loop over the array” beats “; increment esi”. The instruction already says what it does.

Give each register one job

Decide before you write: ESI is the index, EDI is the base. Write it in a comment at the top.

Use meaningful label names

innerLoop and noSwap, not L1 and L2. Labels are free; confusion is not.

Keep blocks short

A dozen instructions doing one clear thing. If it needs a paragraph to explain, split it.

Let C++ do what C++ is good at

Declarations, input and output, string handling. Reserve assembly for the calculation.

Balance every push

Check each path through the block, including the ones a conditional jump creates.

Before you write any assembly, ask

Does this need to be assembly at all?

If the compiler can do it, let it. Optimised C++ usually beats hand-written assembly.

Is there an intrinsic instead?

Compiler intrinsics give you the instruction you want, and still work on x64.

Have I written it in C++ first?

Get it correct in C++, confirm the output, then translate. Now you have a reference to test against.

Can I test the small piece on its own?

A short program that exercises one block is far easier to debug than a large one.

Learning assembly is about understanding the machine.
Using assembly in production is a decision that needs justifying.

A checklist for every _asm block

- [ ] Is the project building for the x86 platform?
- [ ] Did I save EBX, ESI and EDI if I used them?
- [ ] Does every PUSH have a matching POP on every path?
- [ ] Have I left EBP and ESP exactly as I found them?
- [ ] Did I clean up after every CALL — four bytes per argument?
- [ ] Am I using the signed jumps for signed data?
- [ ] Did I zero or sign-extend EDX before a DIV or IDIV?
- [ ] Could ECX be zero when a LOOP begins?
- [ ] Have I commented what each register is being used for?

14

Summary and further reading

What you now know, and where to take it next.



Summary

1. Assembly is a readable name for machine code — one instruction in, one instruction out.
2. MSVC inline assembly lets you write x86 instructions inside a C++ function, using your own variable names.
3. It works on the 32-bit x86 platform only, so every project must be built for x86.
4. Declare data in C++; compute in assembly. Inline assembly cannot define data of its own.
5. Control flow is CMP followed by a conditional jump — signed jumps for signed data.
6. Calls use the stack: arguments pushed right to left, result returned in EAX, caller cleans up.
7. Preserve EBX, ESI and EDI, never touch EBP or ESP, and balance every push with a pop.

Further reading

REFERENCE DOCUMENTATION

- Microsoft Learn — “Inline Assembler” in the Visual C++ documentation
- Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 2 — the instruction reference

TEXTBOOKS

- Irvine, Assembly Language for x86 Processors — the standard undergraduate text
- Patterson & Hennessy, Computer Organization and Design — architecture behind the instructions

WHERE THIS LEADS

x64 assembly

MASM and ml64, or compiler intrinsics — inline assembly does not exist there

Compilers

How a compiler chooses instructions and allocates registers

Reverse engineering

Reading disassembly you did not write: security, malware analysis, debugging

Embedded systems

Where hand-written assembly is still routinely necessary

Before the lab



Set up a project

Create a Win32 console project, switch it to x86, and get the handout example building and running.



Retype example 11.1

Type it rather than pasting it. Typing catches the syntax details that reading skips over.



Practise the debugger

Set a breakpoint inside an _asm block, open the Disassembly and Registers windows, and step through it.



Bring your questions

Write down anything that did not build, with the exact error message.

Every lab exercise builds directly on one of the five examples you have just seen.